

Roll No. _____

Name: _____

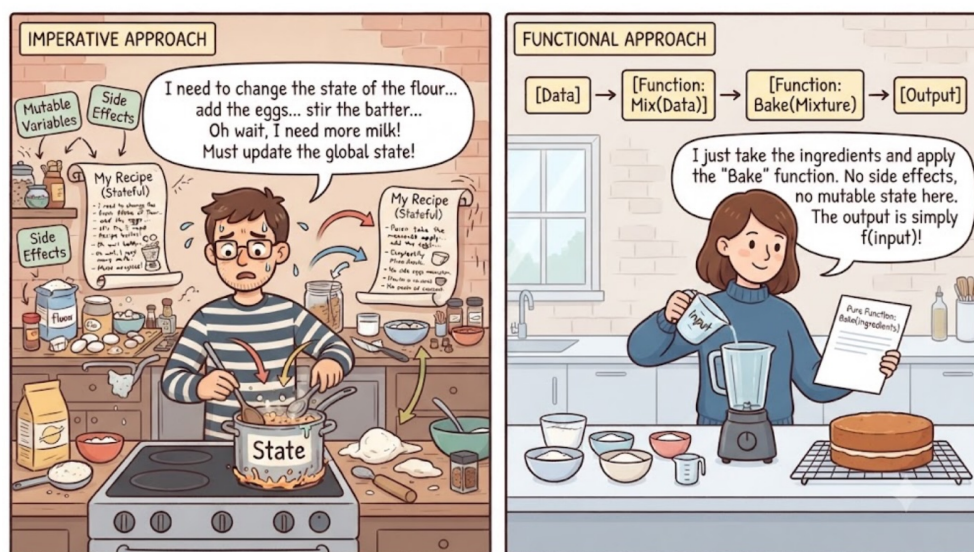
Seat: _____

CS339: Abstractions and Paradigms for Programming

Quiz 1 (50 minutes; 10 marks)

August 27th, 2026

Write neat and crisp answers in the space provided. Space for rough work is provided at the end.



Assume the following are already available to you exactly as defined below, matching what was used in class and in Lab 3 (used in Q6 and Q7):

```
(define (filter pred l)
  (if (null? l) nil
      (let ((x (car l)))
        (if (pred x) (cons x (filter pred (cdr l))) (filter pred (cdr l))))))
```

```
(define (foldr f v l)
  (if (null? l) v (f (car l) (foldr f v (cdr l)))))
```

```
(define (foldl f v l)
  (if (null? l) v (foldl f (f v (car l)) (cdr l))))
```

```
(define (map f l)
  (if (null? l) nil (cons (f (car l)) (map f (cdr l)))))
```

Q1 [1]. Consider the following code:

```
(define (bad) (bad))

(define (second-choice x y) (if (= x 0) 0 y))
(define (first-choice a b) (second-choice a (bad)))
```

(a) [0.5] What does `(first-choice 0 5)` evaluate to under **normal-order** evaluation?

(b) [0.5] What does `(first-choice 0 5)` evaluate to under **applicative-order** evaluation?

Q2 [1] Recall the problem with multiple representations discussed in the class: two representations both defining a procedure named `real-part` caused one definition to silently overwrite the other. Tagging (attaching a symbol like `'rectangular` and dispatching on it) fixes *that* problem. Now consider:

Team A defines a representation for 2D coordinates, tagged `'point`: `(attach-tag 'point (cons x y))`. Completely independently, Team B defines a representation for labeled graph nodes, *also* tagged `'point` (a coincidental naming clash neither team knows about): `(attach-tag 'point (cons label neighbor-list))`.

Team A's code calls their own generic operation, which dispatches on type-tag and — seeing the tag `'point` — runs Team A's point-specific procedure. But the data it's actually handed is one of Team B's `'point`-tagged graph nodes. What happens?

Circle the correct option(s) below:

- (A) The dispatch procedure notices the contents don't match Team A's expected shape and raises a clear "type mismatch" error
- (B) The dispatch matches successfully on the tag alone; Team A's procedure runs anyway, on data it wasn't designed for — likely producing silently wrong output, not a clear type-mismatch error
- (C) Scheme detects that two different `attach-tag` calls used the same tag and raises a redefinition error, the same way the original `real-part` clash did
- (D) The two representations quietly merge into a single combined type, and either team's procedures can correctly operate on either team's data

Q3 [1]. Consider the following code:

```
(define (mystery1 n)
  (if (= n 0)
      1
      (* n (mystery1 (- n 1)))))

(define (mystery2 n)
  (define (helper n acc)
    (if (= n 0) acc (helper (- n 1) (* n acc))))
  (helper n 1))
```

Both compute the same result for a given n .

- (a) [0.5] When evaluating `(mystery1 5)`, how many multiplications are still **pending** (not yet executed) at the moment the base case ($n = 0$) is reached?

- (b) [0.5] When evaluating `(mystery2 5)`, how many multiplications are still **pending** at the moment the base case is reached?

Q4 [2] Consider the following code:

```
(define (make-adder n) (lambda (x) (+ x n)))

(define add5 (make-adder 5))
(define add10 (make-adder 10))
```

What does each of the following evaluate to? [0.5 each]

1. `(add5 3)`

2. `(add10 (add5 3))`

3. `((make-adder 100) 1)`

4. `(add5 (add10 0))`

Q5 [2] Recall the Church boolean encodings:

```
(define ctrue (lambda (t) (lambda (f) t)))
(define cfalse (lambda (t) (lambda (f) f)))
```

(a) [1] Using only `ctrue`, `cfalse`, and function application (no Scheme `if/cond`), define `cor`, which takes two Church booleans and returns a Church boolean that is “true” if **at least one** input is “true.”

(b) [0.5 each] Using your `cor` from (a), what do the following evaluate to ('yes' or 'no')?

1. `((((cor ctrue) cfalse) 'yes) 'no)` _____

2. `((((cor cfalse) cfalse) 'yes) 'no)` _____

Q6 [1] Fill in the two blanks so that `sum-cubes-above-10` correctly returns the sum of the cubes of just the numbers strictly greater than 10 in a given list. (`filter`, `foldr` as defined at the top of this paper.)

```
(define (sum-cubes-above-10 lst)

  (foldr + 0 (map _____

                (filter _____ lst)))))
```

[0.5 mark each blank]

Q7 [1] Using the `foldr/foldl` definitions given at the top of this paper, what does each of the following evaluate to? [0.5 each]

1. `(foldr cons nil (list 1 2 3))` _____

2. `(foldl (lambda (acc x) (cons x acc)) nil (list 1 2 3))` _____

Q8 [1] Consider the following code:

```
(define total 100)
(define (charge! amt)
  (set! total (- total amt))
  total)
```

Recall the ways `set!` may violate evaluation-order assumptions that were valid for purely functional programs.

- (a) [0.5] If, in `(+ (charge! 10) (charge! 20))`, the two calls are evaluated **left to right**, what does the whole expression evaluate to?

- (b) [0.5] If instead they are evaluated **right to left** (the `(charge! 20)` runs first), what does the whole expression evaluate to?

SPACE FOR ROUGH WORK

SPACE FOR ROUGH WORK